클라우드 엔지니어링 6기

멋쟁이사자처럼

26.04.01 (수) | 정범석님

자바 배열, 컬렉션, CMD 인자 정리

👉 오늘의 강의 학습 요약

자바에서 데이터를 저장하는 3가지 방식인 변수, 배열, 컬렉션을 비교하며 저장 개수, 관리 용이성, 저장 가능한 데이터 타입, 크기 변경 가능 여부를 중심으로 특징을 정리했습니다. 특히 배열, 컬렉션이 참조형이라는 점과, 컬렉션에서 기본형을 직접 저장하지 못할 때 래퍼 클래스로 우회한다는 개념을 함께 연결했습니다.

이후 배열 문법을 1차원부터 2차원까지 확장해 선언, 생성, 초기화, 출력(일반 for/향상된 for), 길이 확인, 예외를 체계적으로 다뤘습니다. 마지막으로 `main(String[] args)`의 의미를 커맨드 라인 아규먼트 관점에서 설명하고, 문자열로 들어오는 인자를 `Integer.parseInt()`로 숫자 변환해 반복 출력하는 실습 흐름까지 정리했습니다.

1. 변수, 배열, 컬렉션 비교

> **자바 데이터 저장 방식은 변수, 배열, 컬렉션 3가지이며, 저장 개수/관리 방식/타입 제약/크기 변경/참조형 여부가 핵심 비교 지점입니다.**

자바에서 데이터 저장 방식은 크게 변수, 배열, 컬렉션으로 정리됩니다. 변수는 단일 값 저장에 최적화되어 있으나 데이터 개수만큼 변수가 필요해 관리 부담이 커집니다. 배열과 컬렉션은 "변수 1개로 여러 데이터"를 다루며 관리가 상대적으로 용이합니다.

구분	저장 개수	변수(식별자) 필요 수	저장 데이터 타입 제약	크기 변경	저장소 성격
변수	1개	데이터 개수만큼 필요합니다.	기본형/참조형 모두 저장 가능합니다.	해당 사항이 없습니다.	기본형은 값 자체, 참조형은 주소값을 저장합니다.
배열	여러 개	배열명 1개면 충분합니다.	동일한 타입만 저장 가능합니다.	생성 후 변경 불가합니다.	배열명은 주소값을 저장하는 참조형 변수입니다.
컬렉션	여러 개	컬렉션 참조 1개면 충분합니다.	서로 다른 타입 혼합 저장이 가능합니다.	자동 확장/축소가 가능합니다.	컬렉션 자체는 참조형이며 기본형은 래퍼로 저장합니다.

배열은 동일 타입만 저장 가능하고 크기 변경이 불가하므로, 데이터가 늘어나면 더 큰 배열을 새로 만들어야 합니다. 그럼에도 변수 여러 개를 직접 관리하는 것보다 구조적으로 유리합니다.

컬렉션은 배열 대비 "타입 혼합 가능"과 "동적 크기 변경"이 큰 장점입니다. 다만 컬렉션은 원칙적으로 참조형만 저장하므로, 기본형 값은 래퍼 클래스를 통해 참조형으로 다뤄 저장합니다.

핵심 키워드 #변수 #배열 #컬렉션 #참조형 #크기 변경

2. 래퍼 클래스와 컬렉션 타입

> **래퍼 클래스는 기본형 8개에 대응하는 API 클래스 묶음이며, 컬렉션이 참조형만 저장한다는 제약을 보완합니다.**

컬렉션은 참조형만 저장할 수 있으므로 기본형을 그대로 넣을 수 없다는 제약이 있습니다. 이를 해결하기 위해 자바는 기본형에 대응하는 래퍼 클래스를 제공합니다. 래퍼 클래스는 "기본형 값을 객체(참조형)로 다루기 위한 클래스"이며, 자바가 제공하는 API입니다.

기본형(Primitive)	래퍼 클래스(Wrapper)
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

`Integer` 와 `Character` 는 이름이 기본형과 다르므로 별도로 기억해야 합니다. 나머지는 기본형 이름의 첫 글자를 대문자로 바꾸면 래퍼 클래스가 됩니다.

컬렉션은 저장 방식에 따라 크게 3계열로 구분됩니다. 각 계열은 "순서/중복/검색 방식"이 달라 실제 문제 유형에 따라 선택 기준이 달라집니다.

계열	저장 순서	중복	핵심 특징
List	유지됩니다.	허용됩니다.	인덱스 기반 접근이 가능해 순서가 중요할 때 유리합니다.
Set	유지되지 않습니다.	제거됩니다.	동일 값은 1개만 유지되어 중복 제거에 적합합니다.
Map	키 기반입니다.	값은 키로 관리됩니다.	Key-Value 쌍으로 저장하며 키로 빠르게 검색합니다.

Map은 값을 얻기 위해 키를 사용하므로, 값만 저장하는 List/Set 대비 “검색 속도” 측면에서 장점이 큽니다. 데이터가 많을수록 “키로 바로 찾는 구조”가 효율을 만듭니다.

핵심 키워드 #래퍼 클래스 #Integer #List #Set #Map

3. 1차원 배열 문법과 출력

> **1차원 배열은 동일 타입 데이터의 연속 저장소이며, 선언, 생성, 초기화, 길이, 예외, 출력(for/향상된 for)을 함께 정리해야 합니다.**

배열 선언 문법은 `데이터형 [] 배열명` 형태이며, 대괄호는 배열명 앞/뒤 모두 가능하지만 일반적으로 데이터형 뒤에 작성합니다. 배열명은 참조형 변수로서 "주소값"을 저장하며, 이를 따라가면 실제 배열 영역(힙 메모리)에 접근합니다.

배열 생성은 `new 데이터형 [크기]` 로 수행되며 생성된 배열 요소는 자동 초기화됩니다. 예를 들어 `int` 는 `0`, `String` 은 `null` 로 초기화됩니다.

배열 요소는 인덱스로 접근하며 인덱스는 `0` 부터 시작합니다. 존재하지 않는 인덱스를 접근하면 `ArrayIndexOutOfBoundsException` 예외가 발생하므로, 반복 조건과 인덱스 범위를 항상 일치시켜야 합니다.

배열 길이는 `배열명.length` 로 구하며, 이는 메서드가 아니라 "필드처럼" 괄호 없이 사용합니다. 반면 문자열 길이는 `문자열변수.length()` 로 메서드 호출 형태임을 구분해야 합니다.

대상	길이 구하는 문법	주의점
배열	`arr.length`	괄호를 붙이면 안 됩니다.
문자열	`str.length()`	반드시 `()` 가 필요합니다.

배열 출력은 반복문과 궁합이 좋으며, 일반 `for` 는 조건/증감 실수로 오류가 날 수 있습니다. 이를 줄이기 위해 향상된 `for-each` 가 제공되며, 요소 개수만큼 자동 반복하므로 인덱스 범위 오류 가능성이 낮습니다.

```
int[] nums = {1, 2, 3};
for (int n : nums) {
    System.out.println(n);
}
```

향상된 for는 "배열에서 요소를 하나씩 꺼내 변수에 담아 반복"하는 구조입니다. 반복 횟수, 인덱스 범위를 개발자가 직접 작성하지 않아도 되므로 출력/순회에 자주 사용됩니다.

핵심 키워드 `#배열 선언` `#new` `#인덱스` `#length` `#향상된 for`

4. 1차원 배열 생성 3가지 방법

> **1차원 배열은 new 방식, 리터럴 방식, new+리터럴 혼합 방식으로 생성할 수 있으며 분리 가능 여부와 크기 지정 규칙이 다릅니다.**

1차원 배열 생성은 3가지가 있으며, 특히 "선언과 생성을 분리할 수 있는지"가 차이점입니다. 실무적으로는 리터럴 방식(2번)을 자주 쓰며, 상황에 따라 혼합 방식(3번)을 사용합니다.

방법	예시	선언, 생성 분리	크기 지정
1) new 이용	`int[] a = new int[3];`	가능합니다.	반드시 지정해야 합니다.
2) 리터럴 이용	`int[] a = {1,2,3};`	불가능합니다.	값 개수로 결정됩니다.
3) new+리터럴	`int[] a; a = new int[]{1,2,3};`	가능합니다.	대괄호에 크기를 쓰면 에러입니다.

리터럴 방식은 선언과 동시에 값이 주어지며, 문법적으로 "선언만 해두고 나중에 `{...}` 로 대입"이 허용되지 않습니다. 반면 new+리터럴 방식은 `new int[]{...}` 형태를 사용하면 분리 대입이 가능합니다.

배열은 크기 변경이 불가능하지만, 요소 값의 수정은 인덱스로 접근해 언제든 가능합니다. 따라서 "크기 불변"과 "요소 변경 가능"을 분리해 이해해야 합니다.

핵심 키워드 `#리터럴` `#new int[]{}` `#선언과 생성 분리` `#크기 지정` `#요소 수정`

5. 배열 실습: 합계 누적과 문자열 길이

> **향상된 for로 합계를 누적하고, 문자열 배열에서 `length()` 로 글자 수 조건을 검사해 카운트 및 문자열 연결을 수행할 수 있습니다.**

정수 배열 총합은 누적 변수(`sum`)를 반복문 바깥에 두고, 반복하며 값을 누적하는 패턴으로 해결합니다. 이 방식은 배열 뿐 아니라 컬렉션/입력 처리에서도 매우 자주 쓰이는 기본 패턴입니다.

문자열 배열에서 "두 글자인 이름의 개수"를 구하려면, 각 문자열의 길이를 `s.length()` 로 얻어 조건문으로 검사하고 카운트를 증가시키면 됩니다. 추가로 "누구인지"까지 보고 싶다면, 조건을 만족하는 이름을 별도 저장소에 누적해야 하며 간단히는 문자열 덧셈(연결)으로 누적할 수 있습니다.

작업	핵심 도구	핵심 포인트
합계	누적 변수 + 반복	반복 바깥 초기화와 반복 내부 누적이 분리됩니다.
글자 수 조건	`length()` + `if`	배열 `length` 와 문자열 `length()` 를 혼동하지 않습니다.
대상 이름 모으기	문자열 연결	초기값은 `null` 보다 빈 문자열이 안전합니다.

핵심 키워드 `#누적(sum)` `#향상된 for` `#String.length()` `#카운트` `#문자열 연결`

6. 2차원 배열 구조와 생성, 접근

> **2차원 배열은 "행 배열이 열 배열을 참조하는 구조"이며, 행/열 생성 순서와 접근식 `arr[r][c]` 의 의미를 그림처럼 이해해야 합니다.**

2차원 배열은 흔히 바둑판처럼 설명하지만, 실제 메모리 관점에서는 "2차원 배열(행)이 1차원 배열(열)의 주소를 여러 개 들고 있는 구조"입니다. 즉 `int[][]` 에서 바깥 배열의 각 요소는 `int[]` 를 참조합니다.

2차원 배열 선언은 `데이터형 [][] 배열명` 이며, 행과 열을 모두 만들기 위해 `new` 가 계층적으로 적용됩니다. 먼저 행을 만들고, 각 행에 해당하는 열(1차원 배열)을 별도로 생성해 할당합니다.

단계	코드 개념	결과
1) 선언	`int[][] a;`	참조 변수만 생깁니다.
2) 행 생성	`a = new int[2][];`	행(바깥 배열)만 생성되고 열은 `null` 입니다.
3) 열 생성	`a[0] = new int[2]; a[1] = new int[2];`	각 행이 참조할 열 배열이 생성됩니다.
4) 요소 접근	`a[0][1]`	"0행의 1열" 값에 접근합니다.

2차원 배열의 길이도 "무엇의 길이인지"가 중요합니다. `a.length` 는 행의 개수이며, `a[i].length` 는 i번째 행이 참조하는 열의 개수입니다. 비정방형(가변 길이)에서는 행마다 열 길이가 달라질 수 있으므로 `a[i].length` 를 사용해 안전하게 반복해야 합니다.

핵심 키워드 `#2차원 배열` `#행/열` `#arr.length` `#arr[i].length` `#요소 접근`

7. 2차원 배열 출력과 향상된 for 주의점

> **2차원 배열 출력은 중첩 반복이 필요하며, 향상된 for에서 바깥 반복 변수 타입은 "1차원 배열 타입"이 되어야 합니다.**

2차원 배열 요소 전체를 출력하려면 행을 도는 반복과 열을 도는 반복이 필요하므로 일반적으로 중첩 for를 사용합니다. 일반 for에서는 바깥 조건은 `a.length`, 안쪽 조건은 `a[i].length`로 잡는 것이 핵심입니다.

향상된 for를 사용할 때는 "2차원 배열에서 한 번 꺼내면 1차원 배열이 나온다"는 점을 반드시 지켜야 합니다. 따라서 바깥 반복 변수 타입은 `int[]`가 되어야 하며, 그 다음 안쪽에서 `int`로 요소를 꺼냅니다.

```
for (int[] row : a) {
    for (int v : row) {
        System.out.println(v);
    }
}
```

이 규칙을 이해하면 3차원에서도 동일하게 확장됩니다. 3차원에서 한 번 꺼내면 2차원이 나오므로 바깥 반복 변수는 `int[][]`가 되어야 하는 식으로 차원이 한 단계씩 내려갑니다.

핵심 키워드 `#중첩 for` `#향상된 for` `#int[]` `#차원 감소` `#전체 순회`

8. 2차원 배열: 정방형과 비정방형

> **자바 2차원 배열은 정방형(모든 행의 열 길이 동일)과 비정방형(행마다 열 길이 다름)을 모두 만들 수 있습니다.**

정방형 배열은 모든 행의 열 개수가 동일합니다. 이 경우 `new int[행][열]` 형태로 행과 열을 한 번에 생성할 수 있어 코드가 간결해집니다. 반면 비정방형 배열은 행마다 열 길이가 다르며, 각 행의 열 배열을 서로 다른 크기로 생성해 메모리를 더 효율적으로 쓸 수 있습니다.

유형	생성 방식	특징
정방형	`new int[r][c]`	행/열을 한 번에 생성 가능해 간편합니다.
비정방형	`new int[r][]` 후 각 행에 `new int[각기다른열]`	데이터 구조에 맞춰 열 길이를 절약할 수 있습니다.

비정방형에서는 반복 시 `a[i].length`를 반드시 사용해야 합니다. 열 길이가 모두 같다고 가정하고 고정 크기로 반복하면 인덱스 예외가 발생할 수 있습니다.

핵심 키워드 #정방형 #비정방형 #new int[r][c] #new int[r][] #메모리 효율

9. 2차원 배열 생성 3가지 방법

> **2차원 배열도 1차원과 동일하게 new 방식, 리터럴 방식, new+리터럴 혼합 방식이 있으며 크기 지정 규칙이 핵심입니다.**

2차원 배열 역시 생성 방법이 3가지입니다. 리터럴 방식은 작성이 가장 간단하지만 선언, 생성 분리가 불가합니다. new+리터럴 혼합은 분리 가능하되, 1차원과 동일하게 `new int[행][열]{...}` 처럼 크기를 함께 쓰면 에러가 납니다.

방법	예시	분리 가능	크기 지정 규칙
1) new 이용	`new int[2][2]` 또는 단계 생성	가능합니다.	정방형은 행, 열을 함께 지정 가능합니다.
2) 리터럴	`int[][] a = {{1,2},{3,4}};`	불가능합니다.	값 구조가 크기를 결정합니다.
3) new+리터럴	`a = new int[][]{{1,2},{3,4}};`	가능합니다.	대괄호에 크기를 쓰면 에러입니다.

리터럴로 비정방형을 만들고 싶다면 행마다 다른 길이의 중괄호를 배치하면 됩니다. 예를 들어 `{{1},{2,3},{4,5,6}}` 형태는 3행이며 열 길이가 1,2,3으로 달라집니다.

핵심 키워드 `#int[][]` `#리터럴 2차원` `#new int[][]{}` `#정방형 생성` `#비정방형 리터럴`

10. 메모리 관점: 스택과 힙

> **배열 참조 변수는 스택(로컬)에, new로 생성된 실제 배열은 힙에 저장되며 참조를 통해 접근합니다.**

배열명(참조 변수)이 로컬 변수라면 스택 영역에 저장됩니다. 반면 `new` 로 생성되는 배열 실체(행 배열/열 배열 포함)는 힙에 생성됩니다. 따라서 스택의 참조 변수가 힙 객체를 가리키며, 인덱스 접근을 통해 힙 내부 요소를 읽고 수정합니다.

구성 요소	저장 위치	역할
로컬 변수(배열명)	스택(Stack)	힙의 배열 주소값을 보관합니다.
`new` 로 만든 배열	힙(Heap)	실제 요소가 저장되는 공간입니다.

참조형 데이터에서 "참조가 끊기면 더 이상 접근할 수 없고 관리 대상에서 벗어난다"는 흐름을 함께 이해하면, 배열 선언이 왜 필요한지와 생성 후 참조 유지가 왜 중요한지까지 연결됩니다.

핵심 키워드 #스택 #힙 #참조 변수 #new #주소값

11. main(String[] args)와 커맨드 라인 아규먼트

> **`main(String[] args)`의 args는 실행 시 전달되는 값(아규먼트)을 문자열 배열로 받기 위한 파라미터이며, IDE에서도 Run Configuration으로 주입할 수 있습니다.**

`main(String[] args)`에서 `args`는 문자열 배열 파라미터입니다. 프로그램 실행 시 `java ClassName 값1 값2 ...` 형태로 값을 전달하면, 공백을 구분자로 하여 각 값이 `args` 배열에 저장됩니다.

전달된 값은 모두 문자열로 들어오므로, 숫자를 입력해도 `"10"` 같은 문자열로 처리됩니다. 따라서 숫자 연산이 필요하면 문자열을 숫자로 변환하는 과정이 반드시 필요합니다.

이클립스에서는 CMD에서 실행하는 대신 Run Configuration을 통해 동일한 효과를 낼 수 있습니다. 실행 대상 클래스를 올바르게 선택한 뒤, Arguments 탭의 Program arguments에 값을 공백으로 구분해 입력하면 됩니다.

항목	내용	주의점
전달 단위	공백 기준 토큰	공백이 구분자입니다.
저장 타입	`String[]`	숫자도 문자열로 저장됩니다.
IDE 설정	Run Configuration → Arguments	실행 클래스 선택이 정확해야 합니다.

핵심 키워드 `#main(String[] args)` `#커맨드 라인 아규먼트` `#Run Configuration` `#Program arguments` `#문자열 배열`

12. 문자열 숫자 변환: Integer.parseInt

> **커맨드 라인으로 받은 문자열 숫자는 `Integer.parseInt()` 로 int로 변환해야 반복/연산이 가능하며, 값이 없을 때는 인덱스 접근 예외를 주의해야 합니다.**

커맨드 라인 인자를 숫자로 사용하려면 문자열을 정수로 변환해야 합니다. 이때 사용하는 대표 메서드가 래퍼 클래스 `Integer` 의 `parseInt()` 입니다. `"3"` 을 넣으면 정수 `3` 으로 반환되어 반복 횟수, 계산 등에 사용할 수 있습니다.

```
int n = Integer.parseInt(args[0]);
for (int i = 0; i < n; i++) {
    System.out.println("Hello");
}
```

인자를 전달하지 않았는데 `args[0]` 에 접근하면 배열이 비어 있으므로 예외가 발생합니다. 따라서 실행 전에 반드시 Program arguments에 값이 전달되었는지 확인해야 하며, 클래스 선택이 올바른지도 함께 점검해야 합니다.

문자열 실수 변환은 `Float.parseFloat()` 같은 메서드도 있으나, 강의 흐름에서는 정수 변환이 가장 빈번한 사용 사례로 강조되었습니다. 문자열에 `+ 1` 을 하면 숫자 덧셈이 아니라 문자열 연결이 되므로, 연산 목적이면 변환이 필수입니다.

핵심 키워드 `#Integer.parseInt` `#래퍼 클래스` `#args[0]` `#문자열 숫자 변환` `#반복문`